Too Many Hands on the Throttle: Testing the Interaction of Autonomous Reliability Agents

Abhishek Pandey
Meta
pandeyabhi1987@gmail.com

Abhinav Pandey
Independent Researcher
pandey.aby@gmail.com

Abstract—A production service fleet runs an autoscaler, a load shedder, a circuit breaker, a concurrency limiter, a rate limiter, a timeout policy, a retry budget, and a router. These are separate control loops, owned by separate teams, configured through separate surfaces, and tuned in isolation. They all observe correlated signals and they all act on one plant. We argue that this arrangement can fail in ways that no single controller's telemetry attributes, and we characterize when.

We identify three coupling channels and characterize each. Actuation coupling arises when two actuators move the same plant variable: their gains superpose, and we show that controllers individually tuned inside the delay margin compose to an unstable loop, with the stability condition depending on a quantity no single controller's configuration surface exposes. Observation coupling arises when one controller's action corrupts another's input: we show that when a load-reducing controller acting upstream of a provisioning controller's measurement point trips below the provisioner's utilization target, the composed system has a neutrally stable mode and can settle into persistent under-provisioning, a trap that changing gains or smoothing windows does not escape but reordering the two thresholds does. Topological coupling arises when an action redistributes load, and we give the subcriticality condition under which ejection does not cascade.

We then present Concerto, an arbitration layer with one principle per channel: a single severity estimate with a bounded authority budget allocated rather than summed across actuators, counterfactual reconstruction of each controller's input signal, and topology-gated admission of redistributive actions. An audit of the configuration surfaces of Kubernetes, Envoy, Istio, and Linkerd finds no documented hook linking autoscaling to any protection mechanism, while Istio and Envoy expose narrow intra-proxy guards between ejection, load balancing, and retries.

We report the model, the audit, pre-registered simulator experiments, and a first testbed experiment on unmodified Kubernetes, Envoy, and prometheus-adapter (Section X): with a per-pod shedder tripping below the autoscaler's target, the fleet stayed at its replica floor and served 38% of load, vs. 100% with the thresholds reversed (3 TRAP, 3 REVERSE trials). The trap also reproduces with Envoy adaptive concurrency as the shedder (29% vs. 100%) and with independent shedding at two tiers (50% vs. 99.5%). The remedies fare worse. On the testbed, a static shared-budget stand-in for P1 served 25% of load, worse than independent shedders (50%), because its per-pod budget fell below the autoscaler target and recreated the trap. A topology-gating experiment produced no cascade in any arm, so it could not discriminate. In simulation, the full arbiter lost to a simple retuning. Our main contribution is therefore a tested account of where composed reliability agents fail; the remedy remains unvalidated.

Index Terms—multi-agent interaction, testing autonomous systems, reliability control, autoscaling, load shedding, circuit breaking, control allocation, cascading failure, microservices

I. Introduction

Ask an operator how their service protects itself under load and you will get a list. There is an autoscaler that adds replicas when utilization rises. There is a load shedder that rejects low-priority work when latency climbs. There is outlier detection that ejects unhealthy endpoints. There is a concurrency limiter that caps in-flight requests, a rate limiter that caps outbound calls, a timeout policy, a retry budget, and a router that moves traffic away from trouble.

Each of these is a feedback controller. Each was designed in isolation, documented in isolation, and tuned in isolation. Most are configured through different surfaces, and in a large organization most are owned by different teams. And every one of them observes some function of the same underlying congestion state and acts, directly or indirectly, on that same state.

Control engineers have a name for this situation. When several actuators have authority over one plant, the assignment of corrective effort among them is the control allocation problem, and it has been treated rigorously for decades in aerospace, where an aircraft has more control surfaces than axes of motion [1], [2]. Service fleets have the same structure and do not use the same discipline. We allocate nothing. We run every controller at full authority and hope.

This paper argues that the hope is misplaced in a specific and predictable way, and that the resulting failure modes are invisible from inside any individual controller.

A. Reliability controllers as autonomous agents

Each of these mechanisms is, in the sense used by the agent literature, an autonomous agent: it perceives part of the environment through its own signal, decides on its own policy, and acts without a human in the loop or a protocol with its peers. A fleet is therefore an uncoordinated multi-agent system whose agents were each validated alone. We treat the problem as one of verifying and testing agent interaction: which properties of the composition can be checked statically from configuration, which require execution, and what an arbiter that mediates between agents buys. We do not claim these controllers learn or

plan; the framing concerns autonomy and the absence of coordination, which is exactly what makes interaction failures hard to test for one agent at a time.

B. A failure no controller can see

Consider a fleet with an autoscaler and a load shedder, both configured with defaults that a careful operator would consider reasonable. Load rises. The shedder sees latency climb and begins rejecting requests, which is exactly its job, and latency recovers. The autoscaler, meanwhile, reads request rate at the service, sees it fall, concludes that demand has dropped, and scales the fleet down. Note that this requires an autoscaler configured on a request-rate (custom or external) metric; the Kubernetes default is CPU utilization [10], under which shed requests that never reach the pods have the same effect only if they would have consumed CPU. In our simulation (Section IX) the outcome is not the growing oscillation often described but a stable trap: when the shedder trips below the autoscaler's target, the fleet ratchets down and sheds persistently.

Every controller in this story behaved exactly as specified. The shedder shed under latency pressure. The autoscaler scaled to observed demand. Neither configuration is wrong. The fault is in the composition, and neither controller's telemetry contains enough information to attribute it, because each one sees only its own loop closing correctly.

C. Contributions

1) A taxonomy of three coupling channels (Section III), distinguished by what is shared: the actuated variable, the measured signal, or the load distribution. The distinction matters because each channel calls for a different remedy, and for actuation and observation coupling the safe condition depends on quantities no single controller's configuration exposes.
2) A formal characterization of each channel (Section IV). Proposition 1 shows that controllers individually tuned inside the delay margin lose asymptotic stability once their margin fractions reach unity (with a common delay), and Corollary 1 shows the governing quantity is not observable at any single controller. Proposition 2 shows that signal masking creates a neutral mode and, with threshold misordering, an under-provisioning trap; Corollary 2 identifies which retunings escape it. Proposition 3 gives the subcriticality threshold for redistributive actions.
3) A configuration audit of deployed software (Section V). We audit the configuration surfaces of the mechanisms operators actually run and record, for each, the signal it reads, the plant variable it actuates, and whether any coordination hook with its peers exists. We pair this with the incident record and with the practitioner literature that already names these interactions.
4) Pre-registered composition tests on real components (Section X): the threshold-ordering trap reproduces on Kubernetes and Envoy with three different shedders (12/12 trap trials at the replica floor).
5) Concerto, a proposed arbitration layer (Section VI), with one principle per channel, and its negative evaluation: in simulation it lost to retuning, a testbed stand-in for P1 made attainment worse, and the P3 testbed experiment was uninformative.

We are explicit about scope. Our central evidence is the audit and pre-registered testbed experiments on the observation-coupling trap. The full arbiter is evaluated only in simulation, where it loses to a simple retuning; we report that as a negative result. No claim here concerns production systems.

II. Background

A. The mechanisms

We use the following terms for the controllers a modern fleet runs. An autoscaler adjusts replica count from a utilization or demand signal. A load shedder or admission controller rejects a fraction of offered work, usually prioritized, when a congestion signal exceeds a threshold. A circuit breaker or outlier detector removes an endpoint from the serving set when its error rate or latency exceeds a threshold. A concurrency limiter caps in-flight requests. A rate limiter caps request rate. A timeout policy bounds per-call latency. A retry policy reissues failed calls, typically bounded by a retry budget. A router distributes traffic across endpoints and performs failover.

III. Three Coupling Channels

We distinguish coupling channels by what the controllers share.

A. Actuation coupling

Two controllers exhibit actuation coupling when their outputs move the same plant variable in the same direction.

A concurrency limiter and a rate limiter both bound offered load. A shedder and a rate limiter both bound admitted work. An autoscaler and an outlier detector both change effective serving capacity, in opposite directions. In each case the plant receives the sum of the corrections while each controller was tuned as though it were applying the whole correction alone.

Within a single proxy, Envoy already recognizes one such interaction: its retry budget is configured as a circuit-breaker threshold that bounds concurrent retries to a fraction of active requests [13].

B. Observation coupling

Two controllers exhibit observation coupling when one controller's actuation lies upstream of the other's measurement point, so that the second controller's input is a function of the first controller's output.

The autoscaler and shedder interaction of Section I is the canonical case. Shedding removes work before it reaches the point where request rate is measured, so the provisioning controller reads a demand signal that the protection controller is actively suppressing. The same structure appears whenever a protective mechanism is deployed at the edge of a measurement boundary, which is the normal place to deploy one.

We expect observation coupling to be the least visible of the three. Actuation coupling at least produces a plant that responds too strongly, which an operator may recognize as overshoot. Observation coupling produces a controller that is behaving correctly on false data.

C. Topological coupling

Two controllers exhibit topological coupling when one controller's action redistributes load and thereby changes the state the other controller observes at a different endpoint.

Ejection and failover routing are both redistributive. Removing an endpoint does not remove its traffic; it moves it. If survivors were already near saturation, the redistribution pushes them past the threshold that triggered the first ejection, and the process compounds. Retry amplification is a temporal variant: a timeout reduction converts slow successes into failures, failures into retries, and retries into additional offered load, so a tightening action increases the very quantity it was meant to reduce.

IV. A Model of Overlapping Authority

We model the fleet around an operating point and treat each coupling channel in turn. The models are deliberately minimal. Their purpose is to identify the structural condition under which composition fails, not to predict trajectories.

A. Setup

Let $x(t)$ denote a scalar congestion state at an endpoint, for instance queue depth or latency deviation from setpoint, normalized so that $x = 0$ is the operating point. Let controllers $i = 1, \dots, m$ observe x and apply corrections u_i. Each controller has an actuation delay $\tau_i > 0$ comprising its measurement window, its decision interval, and the physical time for its action to take effect. For an autoscaler τ includes container start time and is often tens of seconds; for a shedder it is milliseconds.

B. Actuation coupling breaks the delay margin

Consider m controllers with overlapping authority, each applying proportional feedback $u_i = -g_i x(t-\tau)$ with common delay τ and gain $g_i > 0$. Acting alone, controller i closes the loop

$$\dot{x}(t) = -g_i\, x(t-\tau) + d(t),$$

which is asymptotically stable if and only if $g_i\tau < \pi/2$, the classical delay margin for first-order feedback with pure delay. Define controller i's margin fraction

$$\rho_i \;=\; \frac{g_i\tau}{\pi/2} \;\in\; (0,1).$$

A controller tuned to consume half its delay margin has $\rho_i = 1/2$.

Proposition 1 (Gain superposition). If the m controllers act concurrently on the same plant variable, the composed loop is

$$\dot{x}(t) = -\Big(\textstyle\sum_{i=1}^{m} g_i\Big)\, x(t-\tau) + d(t),$$

which is asymptotically stable if and only if $\sum_{i=1}^{m} \rho_i < 1$: marginally stable at equality and unstable above it. In particular, any two controllers each tuned to consume half or more of the delay margin compose to a loop that is not asymptotically stable, despite each being stable in isolation.

Proof sketch: Superposition of the corrections gives the stated composed loop, whose stability condition is $(\sum_i g_i)\,\tau < \pi/2$. Dividing by $\pi/2$ yields $\sum_i \rho_i < 1$. ∎

Corollary 1 (Local tuning cannot prevent this). The margin fraction ρ_i is determined by controller i's own configuration and the plant. The stability condition depends on $\sum_j \rho_j$. No configuration surface enumerated in Section V exposes ρ_j for $j \neq i$, nor any proxy for it (Table I). The condition is therefore not evaluable, let alone enforceable, at any individual controller.

Corollary 1 is the practical heart of the actuation-coupling argument. The failure is not that operators tune badly. It is that the quantity governing stability is not exposed anywhere an operator could read it. Adding a second well-tuned controller to a well-tuned system can be unsafe, and nothing in the audited tooling says so.

Unequal delays complicate the algebra without changing the conclusion. When τ_i differ, the composed characteristic equation $s + \sum_i g_i e^{-s\tau_i} = 0$ replaces the single-delay form. Numerically, with ρ_i defined on each controller's own delay, $\sum_i \rho_i < 1$ remains sufficient but becomes conservative: instability onset occurred at $\sum \rho = 1.20$, 1.79, and 2.00 for equal-ρ pairs with delay ratios 1:2, 1:10, and 1:100 (Section IX). Widely separated delays, such as shedder and autoscaler, are therefore the least prone to this channel; it bites hardest between mechanisms of similar speed.

C. Observation coupling induces positive feedback

Now consider two controllers with different roles. A fast controller F reduces offered load with output $u_F \geq 0$, and a slow controller S provisions capacity from a measured

demand signal. Crucially, F acts upstream of S's measurement point, so S observes

$$\hat{\lambda} \;=\; \lambda - u_F$$

rather than the true offered load λ. Let capacity be $\mu = \mu_0 + u_S$ with $u_S = g_S(\hat{\lambda} - \mu)$, let $u_F = g_F\, x$ with $g_F > 0$, and let congestion respond to capacity with sensitivity $\kappa = -\partial x/\partial \mu > 0$.

Proposition 2 (Signal masking). The path $x \to u_F \to \hat{\lambda} \to u_S \to \mu \to x$ has positive sign. Including the direct path by which u_F itself relieves congestion, the linearization $\dot{x} = -g_F x - \delta\mu$, $\dot{\delta\mu} = g_S(-g_F x - \delta\mu)$ has eigenvalues 0 and $-(g_F + g_S)$: the masked operating point is neutrally stable, not unstable. Capacity is not restored toward true demand; any shed level consistent with $\delta\mu = \delta\lambda - g_F x$ is an equilibrium.

Proof sketch: The system matrix $\begin{pmatrix} -g_F & -1 \\ -g_S g_F & -g_S \end{pmatrix}$ has determinant 0 and trace $-(g_F + g_S)$. ∎

Corollary 2 (Threshold-ordering trap). If the load reducer trips at a utilization u_F^* below the provisioner's target u_S^*, admitted load is held near $u_F^*\mu < u_S^*\mu$, so the provisioner reads demand below its target and never scales up (and may scale down). Changing g_F, g_S, or smoothing windows does not alter this fixed point. It is escaped either structurally, by severing the dependence of $\hat{\lambda}$ on u_F (P2), or by retuning the thresholds so that $u_S^* < u_F^*$.

In simulation (Section IX) a shedder tripping at 60% utilization under a 70% autoscaler target held the fleet at 6–8 replicas where 18 were needed, shedding 64–70% of load, across every combination of shedder gain and stabilization window we tried; lowering the autoscaler target to 55% removed the trap entirely. The trap is thus a configuration-ordering hazard that no single controller's surface flags, but it is not immune to retuning.

D. Topological coupling has a branching threshold

Let an endpoint be ejected and its load redistributed over the surviving members of its pool. Let Γ denote the expected number of additional endpoints driven past the ejection threshold by the redistribution from one ejection, given the current load margin distribution across the pool.

Proposition 3 (Cascade subcriticality). Treating ejections as a branching process with mean offspring Γ, the ejection cascade terminates almost surely if and only if $\Gamma < 1$. When $\Gamma > 1$ the cascade reaches the pool's live-endpoint floor with positive probability.

The practical content is that Γ is computable before acting, from the topology and the current per-endpoint headroom, which is what makes topological coupling the most tractable of the three channels. Existing guards (Envoy's max_ejection_percent and panic threshold [11], [12], Istio's minHealthPercent [14]) bound ejection by fixed fractions rather than by Γ.

V. Does the Premise Hold in Practice?

The model predicts trouble under two conditions: overlapping authority, and independent configuration with no shared signal. This section tests whether both conditions hold in widely deployed software. We use three kinds of public evidence and deliberately avoid proprietary incident data, so that every claim here is checkable by a reader.

A. Configuration surface audit

Method: for each mechanism, read the upstream documentation and configuration reference, and record four fields. The signal the mechanism reads. The plant variable it actuates. The tuning parameters it exposes. Whether any documented hook allows it to observe or defer to a peer mechanism. Table I gives the schema and the mechanisms in scope. We audit Kubernetes horizontal pod autoscaling and cluster autoscaling, Envoy outlier detection, circuit breakers, adaptive concurrency, local and global rate limiting, and retry budgets, Istio destination-rule outlier detection, and Linkerd retry budgets.

Findings. (i) Several groups actuate one variable: three mechanisms move the serving set, three bound admitted load, two add retries, and the autoscaler and ejection move capacity in opposite directions. (ii) We found no documented hook linking autoscaling to any protection mechanism. (iii) Contrary to our prior expectation of a uniformly empty column, there are narrow intra-proxy hooks: Envoy's retry budget is a circuit-breaker threshold [13]; Envoy re-admits ejected hosts in panic mode and caps ejections [11], [12]; Istio disables outlier detection when healthy hosts fall below minHealthPercent [14]. (iv) Envoy's adaptive concurrency rejects with 503, a "gateway failure" that can feed consecutive-gateway-failure ejection [11], an observation-coupling path inside one proxy. Our claim is therefore narrowed to: no audited surface coordinates provisioning with protection.

B. Incident catalogue

Public postmortems can show that these interactions occur outside simulation. We use them for existence and mechanism, not for prevalence. This is a deliberate methodological limit. Vendors publish selectively, and most writeups lack the detail needed to diagnose a controller interaction, so any rate computed over published incidents would be an artifact of disclosure policy rather than a property of systems.

Method: rather than inherit a third-party curated list, whose maintenance we cannot depend on, we index primary sources directly. We search vendor incident archives from major cloud and infrastructure providers, apply stated inclusion criteria, and code each admitted incident against the three channels of Section III. Both authors will code independently and report agreement. This catalogue is not yet compiled. The resulting index, with inclusion criteria and per incident coding, is released

TABLE I
Configuration audit (Envoy v1.39.3, Istio 1.31, Linkerd edge-26.10.1, Kubernetes docs as of 2026-10-11). Per-cell source URLs in audit/mechanisms.csv. †Cluster autoscaler row not re-verified.

Mechanism	Reads	Actuates	Coordination hook
K8s HPA	CPU/mem (default); custom/external metrics	replicas	none (accepts any metric)
K8s cluster autoscaler†	pending pods	nodes	none
Envoy outlier detection	5xx, gateway, local-origin errors	serving set	partial: max_ejection_percent; panic threshold
Envoy circuit breakers	in-flight counts	admitted load	hosts retry budget
Envoy adaptive concurrency	latency vs. minRTT	concurrency	none
Envoy local rate limit	request count	admitted load	none
Envoy retry budget	active+pending requests	retries	yes: bounded to % of active requests
Istio outlier detection	5xx, gateway errors	serving set	yes: minHealthPercent gate
Linkerd retries	response status	retries	none (legacy retryRatio)
Linkerd circuit breaking	failure accrual	serving set	none

as an artifact so that the catalogue does not inherit our own maintenance either.

C. Practitioner literature

Independent of any dataset, the interactions we model are already named in the canonical operational literature. The Google SRE treatment of cascading failures describes overload propagation and the role of retries in amplifying it [6]. The Amazon Builders' Library treats retry amplification and load shedding as system-level concerns rather than local ones [7], [8]. We cite this literature as evidence that the phenomena are recognized, and position our contribution as supplying the model and the remedy that it does not.

VI. Concerto: Arbitration by Construction

Concerto is an arbitration layer that sits between the controllers and the plant. Controllers become advisors: they compute desired corrections, and Concerto decides what is actually applied. The name reflects the intent, which is controllers playing from one score rather than each at full volume.

The design has one principle per coupling channel, plus a cross-cutting scheduling rule.

A. P1: bounded shared authority

Addresses actuation coupling.

Concerto computes a single severity estimate $s \in [0, 1]$ from the plant state, once, and derives a total corrective authority budget $B(s)$. Each actuator receives an allocation $\alpha_i(s)$ subject to $\sum_i \alpha_i(s) \leq B(s)$. Allocations are assigned by actuation cost and time constant, not by letting each controller take what it wants.

The structural property is that total loop gain is bounded by B regardless of how many controllers are enabled. Enabling an additional controller redistributes authority rather than adding to it, so the margin-fraction sum of Proposition 1 is bounded by construction and Corollary 1 no longer bites: the governing quantity is computed in one place, by the component that owns it.

This is control allocation in the aerospace sense [1], applied to a plant whose actuators happen to be named autoscaler and shedder.

B. P2: counterfactual signal reconstruction

Addresses observation coupling.

Each controller is presented with the signal it would observe if no other controller were acting. Concretely, the provisioning controller reads $\lambda = \hat{\lambda} + u_F$, the sum of admitted and shed load, rather than admitted load alone. Generalizing, Concerto maintains for each exposed signal an accounting of the corrections applied upstream of its measurement point, and serves the reconstructed pre-intervention value.

By Corollary 2 this removes the trap structurally, independent of threshold ordering; threshold retuning is the alternative. Shedders and rate limiters typically count rejected work in their own telemetry, so we expect the quantity to be available, but we have not measured the integration cost.

C. P3: topology-gated action admission

Addresses topological coupling.

Before applying any redistributive action, Concerto computes the projected cascade gain Γ of Proposition 3 from the current topology and per-endpoint headroom, and admits the action only if the post-action Γ remains below a subcriticality threshold. When the full requested action set would exceed it, the set is scaled back rather than dropped, preserving as much protective value as the threshold allows.

D. P4: time-scale separation

Cross-cutting.

Even under a shared budget, two actuators can chase the same disturbance component. Concerto splits the severity signal by frequency band and assigns bands to actuators by time constant: the fast band drives shedding and concurrency, the slow band drives replica provisioning. An actuator never receives authority over a disturbance component faster than its own actuation delay, which

prevents a slow controller from reacting to transients it cannot affect in time.

VII. Implementation

We target two settings.

The first is a sidecar and controller pair for Kubernetes and Envoy. The severity estimator and allocator run as a cluster-scoped controller. Existing mechanisms are placed in advisory mode where their configuration permits it, and where it does not, they are bounded externally by the allocator setting their limits each interval. Signal reconstruction is implemented by exposing a reconstructed demand metric that the autoscaler consumes in place of the raw one, which requires no change to the autoscaler itself.

The second is a reference implementation inside a stdlib-Python fleet simulator (released with this paper), which is the only implementation evaluated here. The Kubernetes/Envoy implementation described above is planned, not built.

A deliberate design constraint is that Concerto requires no modification to the controllers it arbitrates. Each of the three principles is implementable by bounding a controller's output, substituting its input, or gating its action, all of which are available from outside.

VIII. Evaluation Protocol

This section states the full campaign. Section IX reports the simulator portion; the testbed portion is pending.

A. Testbed

An open microservice benchmark on Kubernetes with Envoy or Istio providing outlier detection, retries, and concurrency limiting, and horizontal pod autoscaling providing capacity. We favor DeathStarBench as the primary application for its established use in the systems community [9], with a lighter alternative retained for iteration speed. Workloads are driven by an open-loop generator, with arrival patterns taken from public cluster traces to obtain realistic non-stationarity.

The first testbed experiment (Section X) targets the observation-coupling trap of Corollary 2.

B. Simulator

A fleet simulator supplies controlled ablation across many scenarios and at scales the testbed cannot reach. It is the instrument for isolating mechanism, not for establishing that the phenomenon is real; the testbed does that.

C. Baselines

B1, independent defaults. Every mechanism enabled with vendor default configuration. This is not a strawman. It is what a team gets by following each mechanism's own documentation.

B2, independently retuned. A fixed, pre-registered retuning without any shared signal: load-reducer gains scaled by 0.4 and the autoscaler utilization target lowered from 0.70 to 0.55. In the testbed the same rule will be applied, chosen before measurement. This baseline tests whether the problem is a tuning problem.

B3, strict precedence arbiter. A naive arbitration scheme in which a fixed priority order determines which controller acts when several wish to. This isolates the value of proportional allocation over simple mutual exclusion.

B4, Concerto.

D. Metrics and research questions

Metrics: oscillation amplitude and period after a step disturbance, settling time, objective attainment, capacity cost, cascade incidence, goodput under a retry storm, and the rate of simultaneous tightening actions on one endpoint.

RQ1. Does each coupling channel reproduce on unmodified deployed software under reasonable configuration?

RQ2. Does retuning remove instability from actuation coupling but not the observation-coupling deficit? This was our sharpest pre-registered prediction; in simulation it failed (Section IX).

RQ3. Does Concerto restore stability without a cost or attainment penalty relative to the best-performing baseline?

RQ4. What does each principle contribute? Measured by removing P1 through P4 individually.

RQ5. How does arbitration behave as the number of enabled mechanisms grows? Proposition 1 predicts the independent baselines degrade with m while Concerto does not.

IX. Simulator Results

All numbers come from the released simulator (deterministic; 20 seeds for fleet experiments, 30 for pool experiments; 95% normal-approximation CIs, paired by seed). Hypotheses and thresholds were committed in PREREG.md before the runs. These are model checks, not evidence about deployed systems.

Fleet model. An autoscaler (15 s period, 30 s start delay, 300 s downscale stabilization, 70% target) and two integral load reducers (shedder and concurrency limiter, 3 s measurement delay, each tuned alone to $\rho = 0.6$) both trip at 60% utilization. That ordering is a modeling assumption chosen to exercise Corollary 2; we have not shown it is common. Load steps from 80 to 120 req/s with seeded bursts and noise. Attainment is the fraction of offered load admitted while queue depth stays within an SLO bound.

RQ3 (failed). Concerto does not match the best baseline: B2 attains 0.993 vs. 0.916 (-7.7 pp, CI $[-7.9, -7.5]$), though at $1.46\times$ the capacity cost. Concerto still sheds because P2 provisions to the 70% target while both reducers trip at 60%; arbitration does not correct threshold ordering.

TABLE II
Fleet simulator, 20 seeds, mean [95% CI].

Config	Attain.	Cost (repl.)	Osc.
B1 defaults	.532 [.528,.536]	11.4	.220
B2 retuned	.993 [.992,.994]	28.0	.029
B3 precedence	.572 [.570,.575]	12.0	.073
B4 Concerto	.916 [.914,.917]	19.2	.095
less P1	.911 [.910,.912]	19.2	.116
less P2	.348 [.346,.350]	7.3	.040
less P4	.956 [.954,.958]	22.8	.072

RQ2 (failed). Retuning both suppresses oscillation (−0.19 vs. B1) and escapes the observation-coupling trap. The prediction that retuning cannot fix observation coupling is not supported.

RQ4. Removing P2 collapses attainment to 0.348; removing P1 raises oscillation by 0.022 [0.021,0.023]; removing P4 adds 9.05 replica changes but raises attainment to 0.956, so P4 trades responsiveness for stability. P3 is not exercised in the fleet model.

P1 in isolation. With $m = 1..5$ controllers each at $\rho = 0.6$ on a common delay, independent composition grows without bound for every $m \geq 2$; the shared budget ($\sum \rho \leq 0.5$) settles in 5.77 s for every m, faster than the single unconstrained controller (7.61 s).

P3. In a 50-host pool with one faulty host and per-host utilization spread ±0.1 around a mean u, no policy differs for $u \leq 0.85$. At $u = 0.9$ and 0.95 ungated ejection collapses the pool; goodput is 0.999 and 0.986 under the Γ gate vs. 0.973 and 0.925 with a 10% ejection cap and 0.969 and 0.955 with a 50% minHealthPercent gate. Predicted $\Gamma \geq 1$ matched realized cascades in 180/180 ungated runs, but outcomes were bimodal, so this is weak evidence of predictive power. The pool model lets an overloaded host serve at capacity, which favors withholding ejection.

X. Testbed: the Threshold-Ordering Trap

Setup. Single-node k3s v1.31.4 on one 8-vCPU VM. A Python service has a fixed capacity of about 50 req/s per pod (4 workers, 80 ms service time). Each pod carries an Envoy v1.31.4 sidecar whose local_ratelimit filter (token bucket refilled at R req/s per pod, excess answered 429) is the per-pod shedder. An edge Envoy round-robins over pods. The autoscaling/v2 HPA scales on per-pod 2xx rate at the sidecar, served by Prometheus and prometheus-adapter, with a target of 35 req/s (0.7 of capacity), 2–8 replicas, and the default stabilization. Open-loop load from vegeta: 50 req/s for 60 s, then 160 req/s for 480 s. TRAP sets $R = 30$ (trips at 0.6, below target); REVERSE sets $R = 40$ (0.8, above target). Thresholds T1–T3 were committed before any run (PREREG.md v2). Metrics cover the final 240 s.

Result. Every TRAP trial held at the 2-replica floor while offered load was 3.2× the starting load; the HPA read 30 req/s per pod against its 35 req/s target and never scaled. Mean attainment was 0.375 (TRAP) vs. 1.000 (REVERSE), a gap of 0.625. Pre-registered verdicts: T1 PASS, T2 PASS, T3 PASS. The trap therefore reproduces on unmodified, documented components, and, as in simulation, reordering the thresholds escapes it.

TABLE III
Testbed RQ1, final 240 s of each trial. Attainment is the 2xx fraction of offered load; 5 replicas are needed at 160 req/s.

Trial	Attain.	429 frac.	Final repl.	Mean repl.
TRAP 1	0.375	0.625	2	2.00
TRAP 2	0.375	0.625	2	2.00
TRAP 3	0.375	0.625	2	2.00
REVERSE 1	1.000	0.000	5	5.00
REVERSE 2	1.000	0.000	5	5.00
REVERSE 3	1.000	0.000	5	5.00

TABLE IV
Campaign, mean attainment (original-rule valid trials).

Arm	n	Attain.	Final repl.
AC trap (L=2)	3	0.286	2
AC reverse (L=6)	3	1.000	5
Two-tier safe (edge off)	2	0.995	5
Two-tier indep. (edge 80)	3	0.500	3
P1 budget (edge 50 + side 25)	3	0.250	2
P3 none / cap10 / cap50 / Γ	3 each	0.997	–

Caveats. A per-pod token bucket stands in for a capacity-relative shedder; Envoy's adaptive concurrency was not tested. The box kernel lacks the iptables statistic match, so kube-proxy could not balance a multi-endpoint Service; the edge Envoy replaces it. One node, one application, and 3/3 (TRAP/REVERSE) trials per arm.

A. Campaign: three more shedders, P1, and P3

Pre-registered as PREREG v3b before any run. An earlier v3a (dynamic Γ controller on a heterogeneous pool; an allocator over adaptive concurrency, rate limit and retries) was superseded before execution and remains untested. Load and HPA settings match RQ1. P3 instead offers 200 req/s for 180 s to 10 endpoints, one of them always failing. Under the original validity rule (sample gap ≤10 s), one safe two-tier trial was invalid, leaving n=2 for that arm. A relaxed rule added mid-campaign (gap ≤600 s, after kubectl stalls) admits it and changes no verdict; we report the original rule as primary.

Verdicts. AC1–3 PASS: with Envoy adaptive concurrency pinned to a fixed limit as the shedder, the trap reproduces. TT1–3 PASS: independent edge and sidecar shedding halves attainment. P1a–c FAIL. The budget stand-in sets $\Sigma R = 100$ req/s against 160 offered, and its per-pod share (25) is below the HPA target (35). The autoscaler therefore never scaled (2 replicas in every trial), and attainment fell to 0.25. The stand-in only lowers configured rates and does not redistribute authority by severity, so this tests a naive budget, not P1 as

designed. It nonetheless shows that a budget ignoring threshold ordering recreates the observation-coupling trap. P3a FAIL, P3b–c PASS. One failing endpoint caused about 0.3% errors and no cascade under any policy, so all four arms are indistinguishable. The Γ arm was a static cap equal to cap10 at N=10. P3 is uninformative on the testbed, and its only support remains the simulator.

XI. Related Work

Control allocation and feedback computing. Assigning corrective effort among redundant actuators is well developed in aerospace [1], [2], and feedback control of computing systems is treated systematically by Hellerstein et al. [15]. We borrow the allocation view and add two channels the aerospace formulation lacks: actuators that corrupt each other's sensors, and actions that redistribute load across a graph.

Autoscaling. Autoscaling is studied as a control and prediction problem [3]; production systems such as Autopilot [16] and FIRM [17] tune resources from observed signals. These works assume the autoscaler's input reflects demand. Corollary 2 and our testbed show when a co-located shedder breaks that assumption.

Overload and admission control. Server overload control has a long lineage [4]. DAGOR coordinates load shedding across WeChat's microservice graph [18], and Breakwater uses credits to admit RPCs at microsecond scale [19]. Adaptive concurrency limits import congestion control into the application layer [5]. These systems coordinate one mechanism across services. We study uncoordinated different mechanisms acting on one service.

Metastable failures and retry storms. Bronson et al. define metastable failures, in which a trigger pushes a system into a bad state sustained by its own feedback (often retries) [20]. Huang et al. find such failures across production incidents [21]. Retry amplification is a canonical practitioner concern [6], [7]. Our threshold-ordering trap is related but distinct. It is a stable equilibrium of two individually correct controllers and needs no trigger beyond a load step. It persists because each agent's own loop closes correctly.

Cascading failure. The branching threshold of Proposition 3 is standard in cascade models. Our use is as an admission gate on an agent's action; Envoy's ejection cap and Istio's minHealthPercent are fixed-fraction precursors [11], [14].

Testing agent interaction. Agents validated alone can fail together. Our contribution to this venue is a test method of three parts: a static audit of what each agent senses and actuates, pre-registered composition tests on real components, and an arbiter evaluated against an honest baseline.

XII. Threats to Validity

Construct validity. Attainment (2xx over offered) and replica count are proxies for user-facing objectives. Envoy's local_ratelimit per pod stands in for a capacity-relative shedder. The adaptive-concurrency variant is part of the pending campaign.

Internal validity. Thresholds were pre-registered (PREREG.md) before each run. Trials with sampling gaps above 10 s or truncated load phases are discarded and rerun, and we report the one such rerun. The box hosting the testbed paused and was once rolled back to a snapshot; the gap check exists for this reason. The Γ controller includes a disclosed exception that allows ejecting a host with under 10% success.

External validity. One node, synthetic services, and two or three mechanisms at a time. Production fleets are larger, heterogeneous, and configured differently. The simulator covers scale only in a model sense.

Conclusion validity. Testbed trials number two or three per arm. The effects are large and consistent (attainment 0.375 vs. 1.000 in every trial), but small n precludes fine-grained estimates. Simulator CIs are normal approximations over 20–30 seeds.

Negative results. The full arbiter lost to expert retuning in simulation (RQ2/RQ3). We report this rather than tune the baseline down.

XIII. Artifact Availability

The simulator, audit table with per-cell source URLs, testbed manifests, trial scripts, raw replica traces, analysis code, and the pre-registrations (PREREG.md v1–v3, committed before the corresponding runs) are public at https://github.com/pandeyaby/CONCERTO. The testbed is reproducible on a single VM at no cost (testbed/setup.sh).

XIV. Conclusion

Service fleets run many reliability controllers over one plant and allocate authority among them not at all. We identified three channels through which those controllers interfere, and showed that each has a distinct structural signature: summed gains that break a delay margin no controller can observe, a masked signal that creates an under-provisioning trap when thresholds are misordered, and a redistribution process with a branching threshold. We presented an arbitration layer with one principle per channel, designed to work without modifying the controllers it governs.

The failure side of this account is now tested. The audit found no surface coordinating provisioning with protection. The threshold-ordering trap reproduced on unmodified Kubernetes and Envoy in all 12 pre-registered trap trials across three shedder types, and reversing the ordering escaped it every time. The remedy side is not. In simulation the full arbiter lost to retuning. On the testbed a naive shared budget made things worse by recreating the trap, and our cascade experiment never cascaded. A shared authority budget is only safe if it respects the threshold ordering it is meant to protect; making that constraint part of P1, and building a pool that actually cascades, is the next step.

References

[1] W. C. Durham, "Constrained control allocation," Journal of Guidance, Control, and Dynamics, vol. 16, no. 4, pp. 717–725, 1993.

[2] T. A. Johansen and T. I. Fossen, "Control allocation: A survey," Automatica, vol. 49, no. 5, pp. 1087–1103, 2013.

[3] T. Lorido-Botrán, J. Miguel-Alonso, and J. A. Lozano, "A review of auto-scaling techniques for elastic applications in cloud environments," Journal of Grid Computing, vol. 12, no. 4, pp. 559–592, 2014.

[4] M. Welsh and D. Culler, "Adaptive overload control for busy internet servers," in USENIX Symposium on Internet Technologies and Systems (USITS), 2003.

[5] Netflix Technology Blog, "Performance under load: Adaptive concurrency limits," 2018. [Online]. Available: https://netflixtechblog.medium.com/performance-under-load-3e6fa9a60581

[6] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media, 2016, ch. 22, "Addressing Cascading Failures."

[7] M. Brooker, "Timeouts, retries, and backoff with jitter," Amazon Builders' Library. [Online]. Available: https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

[8] D. Yanacek, "Using load shedding to avoid overload," Amazon Builders' Library. [Online]. Available: https://aws.amazon.com/builders-library/using-load-shedding-to-avoid-overload/

[9] Y. Gan et al., "An open-source benchmark suite for microservices and their hardware-software implications for cloud and edge systems," in ASPLOS, 2019.

[10] Kubernetes, "Horizontal pod autoscaling." [Online]. Available: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/, accessed Oct. 2026.

[11] Envoy v1.39.3, "Outlier detection." [Online]. Available: https://www.envoyproxy.io/docs/envoy/v1.39.3/intro/arch_overview/upstream/outlier

[12] Envoy v1.39.3, "Panic threshold." [Online]. Available: https://www.envoyproxy.io/docs/envoy/v1.39.3/intro/arch_overview/upstream/load_balancing/panic_threshold

[13] Envoy v1.39.3, "Circuit breakers (config.cluster.v3)." [Online]. Available: https://www.envoyproxy.io/docs/envoy/v1.39.3/api-v3/config/cluster/v3/circuit_breaker.proto

[14] Istio 1.31, "Destination rule." [Online]. Available: https://istio.io/v1.31/docs/reference/config/networking/destination-rule/

[15] J. L. Hellerstein, Y. Diao, S. Parekh, and D. M. Tilbury, Feedback Control of Computing Systems. Wiley-IEEE Press, 2004, doi:10.1002/047166880X.

[16] K. Rzadca et al., "Autopilot: Workload autoscaling at Google," in Proc. EuroSys, 2020, doi:10.1145/3342195.3387524.

[17] H. Qiu, S. S. Banerjee, S. Jha, Z. T. Kalbarczyk, and R. K. Iyer, "FIRM: An intelligent fine-grained resource management framework for SLO-oriented microservices," in Proc. USENIX OSDI, 2020.

[18] H. Zhou et al., "Overload control for scaling WeChat microservices," in Proc. ACM SoCC, 2018, doi:10.1145/3267809.3267823.

[19] I. Cho, A. Saeed, J. Fried, S. J. Park, M. Alizadeh, and A. Belay, "Overload control for μs-scale RPCs with Breakwater," in Proc. USENIX OSDI, 2020.

[20] N. Bronson, A. Aghayev, A. Charapko, and T. Zhu, "Metastable failures in distributed systems," in Proc. HotOS, 2021, doi:10.1145/3458336.3465286.

[21] L. Huang et al., "Metastable failures in the wild," in Proc. USENIX OSDI, 2022.